عنوان پروژه

دستیار هوشمند آشپزی — Yumio

تهیه‌کننده: مهلا جراحی

شماره دانشجویی: 4011277086

درس: مبانی رایانش ابری

استاد حل تمرین: امیرآبرون

تاریخ: آبان ۱۴۰۵

Scanned with CamScanner

۱. معرفی پروژه

یک دستیار هوشمند آشپزی است که با استفاده از هوش مصنوعی به کاربران کمک می‌کند با توجه به Yumio مواد غذایی موجود در خانه، غذای مناسبی انتخاب کنند.

بسیاری از افراد هنگام آشپزی با این مسئله روبه‌رو هستند که نمی‌دانند با مواد اولیه‌ای که در اختیار دارند چه غذایی می‌توانند تهیه کنند. هدف اصلی این پروژه طراحی رباتی است که این مشکل را از طریق یک گفت‌وگوی ساده در تلگرام برطرف کند.

با استفاده از مدل زبانی هوش Yumio کاربر می‌تواند مواد غذایی موجود در خانه را برای ربات ارسال کند. سپس مصنوعی، اطلاعات را تحلیل کرده و غذاهای مناسب را پیشنهاد می‌دهد.

علاوه بر این، کاربر می‌تواند شرایط دیگری مانند زمان در دسترس، نوع غذا، سبک یا کامل بودن وعده و محدودیت‌های غذایی خود را نیز اعلام کند.

۲. اهداف پروژه

اهداف اصلی طراحی و پیاده‌سازی Yumio عبارت‌اند از:

- طراحی یک دستیار هوشمند آشپزی در محیط تلگرام.
- استفاده از هوش مصنوعی برای تحلیل درخواست‌های متنی کاربران.
- پیشنهاد غذا بر اساس مواد اولیه موجود.
- توجه به محدودیت زمانی و سلیقه غذایی کاربر.
- ارائه دستور پخت مرحله‌به‌مرحله برای غذاهای انتخاب‌شده.
- آشنایی عملی با مفاهیم رایانش ابری و اجرای کد در محیط Cloudflare Workers.
- استفاده از سرویس هوش مصنوعی ابری بدون نیاز به نصب و اجرای مدل روی رایانه شخصی.

۳. قابلیت‌های ربات

قابلیت‌های در نظر گرفته‌شده برای نسخه فعلی Yumio عبارت‌اند از:

۳.۱. شروع و معرفی ربات

ربات پیام خوشامدگویی نمایش می‌دهد و نحوه استفاده از آن را توضیح می‌دهد، با ارسال دستور /start.

۳.۲. راهنمای استفاده

کاربر راهنمای استفاده از ربات و نمونه پیام‌های قابل ارسال را دریافت می‌کند، با ارسال دستور /help.

۳.۳. پیشنهاد غذا بر اساس مواد اولیه

کاربر مواد غذایی موجود در خانه را وارد می‌کند و ربات سه غذای متناسب با آن مواد پیشنهاد می‌دهد.

برای مثال:

«من سیب‌زمینی، تخم‌مرغ و گوجه‌فرنگی دارم. چه غذاهایی می‌توانم درست کنم؟»

ربات باید غذاهای مناسب، مواد اولیه موردنیاز، زمان تقریبی آماده‌سازی و توضیح کوتاهی درباره هر غذا ارائه کند.

۳.۴. پیشنهاد غذا با توجه به زمان

کاربر می‌تواند زمان در دسترس خود را مشخص کند.

برای مثال:

«فقط ۱۵ دقیقه وقت دارم. با تخم‌مرغ و نان چه غذایی درست کنم؟»

Scanned with CamScanner

۳.۵. توجه به سلیقه غذایی
کاربر می‌تواند نوع غذای موردعلاقه خود را مشخص کند؛ برای مثال غذای ایرانی، فرنگی، سبک، کامل یا بدون گوشت.

۳.۶. توجه به حساسیت و محدودیت غذایی
کاربر می‌تواند حساسیت یا محدودیت غذایی خود را اعلام کند تا مدل هنگام پیشنهاد غذا آن را در نظر بگیرد.

۳.۷. ارائه دستور پخت
پس از دریافت پیشنهادها، کاربر می‌تواند نام غذا یا یکی از پیشنهادها را انتخاب کند و از ربات بخواهد روش تهیه آن را به‌صورت مرحله‌به‌مرحله توضیح دهد.
نکته فنی: این قابلیت در نسخه فعلی به توانایی مدل در درک متن جدید وابسته است. چون تاریخچه مکالمات در پایگاه داده ذخیره نمی‌شود، ممکن است ربات نتواند به پیشنهادهای پیام قبلی ارجاع دهد. در چنین شرایطی کاربر باید نام غذا یا مواد اولیه را دوباره ذکر کند.

۴. فناوری‌ها و ابزارهای استفاده‌شده

۴.۱. Telegram
تلگرام محیط ارتباطی میان کاربر و ربات است. کاربر بدون نیاز به نصب نرم‌افزار جداگانه می‌تواند از طریق حساب تلگرام خود با Yumio ارتباط برقرار کند.
برای ساخت ربات از سرویس BotFather استفاده شده است.
وظایف تلگرام در پروژه:
- دریافت پیام‌های کاربران
- ارسال درخواست‌های کاربران به آدرس وب‌هوک
- نمایش پاسخ‌های ربات
- مدیریت دستورات اولیه مانند /help و /start

۴.۲. Cloudflare Workers
Cloudflare Workers بستر اجرای کدهای پروژه در فضای ابری است.
کد به‌جای اینکه برنامه روی رایانه شخصی اجرا شود، روی زیرساخت Cloudflare اجرا می‌شود.
وظایف Worker در پروژه:
- دریافت درخواست‌های ارسالی از تلگرام
- پردازش پیام کاربر
- ارتباط با مدل هوش مصنوعی
- دریافت پاسخ مدل
- ارسال پاسخ به تلگرام

در این پروژه از Worker با نام broken-tree-3537 استفاده شده است.
آدرس Worker:
https://broken-tree-3537.minahp50.workers.dev/

۴.۳. Cloudflare Workers AI
Workers AI سرویس هوش مصنوعی Cloudflare است که امکان اجرای مدل‌های هوش مصنوعی را در

Scanned with CamScanner

زیرساخت ابری فراهم می‌کند.
در این پروژه، مدل زبانی برای تحلیل پیام کاربران و تولید پیشنهادهای غذایی استفاده می‌شود.
انجام شده است AI با نام Binding به این سرویس از طریق یک Worker اتصال
در کد، مدل با استفاده از دستور زیر فراخوانی می‌شود:

```
const result = await env.AI.run(
  "@cf/meta/llama-3.1-8b-instruct-fast",
  {
    messages: [
      {
        role: "system",
        content: systemPrompt
      },
      {
        role: "user",
        content: userText
      }
    ]
  }
);
```

این قسمت مهم‌ترین بخش هوش مصنوعی پروژه است.

۴.۴. JavaScript

زبان برنامه‌نویسی استفاده‌شده برای پیاده‌سازی منطق ربات JavaScript است.
از این زبان برای دریافت پیام، بررسی دستورات، ارسال درخواست به مدل هوش مصنوعی و ارتباط با Telegram API استفاده شده است.

۴.۵. Telegram Bot API

این رابط برنامه‌نویسی امکان ارتباط برنامه با ربات تلگرام را فراهم می‌کند.
در پروژه از دو متد اصلی استفاده شده است:

- sendMessage: برای ارسال پیام متنی به کاربر
- sendChatAction: برای نمایش وضعیت تایپ کردن ربات

همچنین از قابلیت Webhook برای دریافت پیام‌های جدید استفاده شده است.

۵. معماری کلی سیستم

معماری Yumio از چند بخش اصلی تشکیل شده است:

۱. کاربر در تلگرام پیامی برای ربات ارسال می‌کند.

۲. پروژه ارسال می‌کند Webhook تلگرام پیام را به آدرس.

۳. Cloudflare Worker درخواست را دریافت و پردازش می‌کند.

۴. اگر پیام شامل دستور /start یا /help باشد، پاسخ از پیش تعریف‌شده ارسال می‌شود.

Scanned with CamScanner

۵. در غیر این صورت، پیام کاربر به مدل هوش مصنوعی ارسال می‌شود.
۶. مدل هوش مصنوعی درخواست را بر اساس دستورالعمل تعیین‌شده تحلیل می‌کند.
۷. بازگردانده می‌شود Worker پاسخ تولیدشده به.
۸. برای کاربر ارسال می‌کند Telegram Bot API پاسخ را از طریق Worker.

نمودار معماری سیستم:

User
↓
Telegram Bot
↓
Webhook
↓
Cloudflare Worker
↓
Message Handler
↓
Workers AI Binding
↓
Language Model
↓
AI Response
↓
Telegram Bot API
↓
User

۶. نحوه پیاده‌سازی پروژه

۶.۱. ایجاد ربات در تلگرام

ابتدا وارد BotFather شده و با استفاده از دستور /newbot یک ربات جدید ایجاد شد.
پس از تعیین نام و نام کاربری، BotFather یک توکن اختصاصی در اختیار برنامه قرار می‌دهد.
این توکن برای احراز هویت درخواست‌های برنامه به Telegram Bot API استفاده می‌شود.

۶.۲. ایجاد Cloudflare Worker

در داشبورد Cloudflare وارد بخش Workers & Pages شده و یک Worker جدید ایجاد شد.
پس از ایجاد Worker، یک آدرس اینترنتی اختصاصی در اختیار پروژه قرار گرفت.
این آدرس به‌عنوان مقصد Webhook تلگرام استفاده می‌شود.

۶.۳. تنظیم AI Binding

برای برقراری ارتباط میان Worker و سرویس هوش مصنوعی، وارد تنظیمات Worker شده و از قسمت

Scanned with CamScanner

Bindings یک اتصال AI ایجاد شد.
نام این Binding برابر AI انتخاب شد.
برای اجرای مدل استفاده کرد env.AI.run() به همین دلیل در کد می‌توان از دستور.

۶.۴. تعریف Secret

توکن ربات تلگرام نباید مستقیماً در کد برنامه نوشته شود.
Variables and Secrets برای نگهداری امن‌تر آن، از بخش Worker در تنظیمات استفاده شد.
نام متغیر:

TELEGRAM_BOT_TOKEN

مقدار آن همان توکن اختصاصی دریافت‌شده از BotFather است.
env.TELEGRAM_BOT_TOKEN در زمان اجرا، برنامه از طریق به این مقدار دسترسی پیدا می‌کند.

۶.۵. پیاده‌سازی Webhook

Webhook روشی برای دریافت خودکار پیام‌های جدید تلگرام است.
در این روش لازم نیست برنامه دائماً تلگرام را بررسی کند؛ تلگرام هنگام دریافت پیام جدید، یک درخواست HTTP از نوع POST به آدرس Worker ارسال می‌کند.
آدرس Webhook پروژه:

https://broken-tree-3537.minahp50.workers.dev/

تلگرام پیام‌های جدید را به این آدرس ارسال می‌کند، Webhook پس از تنظیم موفق.

۷. بررسی ساختار کد برنامه

کد اصلی پروژه در قالب یک Worker نوشته شده است.
در ادامه بخش‌های مهم آن توضیح داده می‌شوند.

۷.۱. تعریف نقطه ورود برنامه

```
export default {
  async fetch(request, env) {
```

تابع fetch نقطه ورود اصلی برنامه است.
هر زمان درخواستی به Worker ارسال شود، این تابع اجرا می‌شود.
دو پارامتر اصلی آن عبارت‌اند از:

- request: اطلاعات درخواست دریافتی
- env: شامل Bindingهایی مانند AI و متغیرهای محیطی و توکن تلگرام

۷.۲. بررسی درخواست GET

```
if (request.method === "GET") {
  return new Response("Yumio AI is running! 🔍");
}
```

این بخش برای آزمایش فعال بودن Worker استفاده می‌شود.
وقتی آدرس Worker مستقیماً در مرورگر باز شود، پیام فعال بودن سرویس نمایش داده می‌شود.

Scanned with CamScanner

۷.۳. بررسی نوع درخواست

```
if (request.method !== "POST") {
  return new Response("Method not allowed", { status: 405 });
}
```

پذیرفته نمی‌شوند POST ارسال می‌کند، درخواست‌های غیر از POST از آنجا که تلگرام پیام‌ها را با درخواست.

۷.۴. دریافت اطلاعات پیام

```
const update = await request.json();
const message = update.message;
```

دریافت می‌شود JSON اطلاعات ارسالی تلگرام در قالب.

از اطلاعات درخواست استخراج می‌شود message سپس بخش.

۷.۵. استخراج شناسه کاربر و متن پیام

```
const chatId = message.chat.id;
const userText = message.text.trim();
```

در این قسمت:

- chatId شناسه گفت‌وگوی کاربر است.
- userText متن ارسال‌شده توسط کاربر است.

شناسه گفت‌وگو برای ارسال پاسخ به همان کاربر استفاده می‌شود.

۷.۶. تابع ارسال پیام

```
async function sendMessage(text) {
  return fetch(
    `https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}/sendMessage`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        chat_id: chatId,
        text: text
      })
    }
  );
}
```

این تابع برای ارسال پیام به تلگرام طراحی شده است.

ورودی تابع متن موردنظر برای ارسال است.

ارسال می‌شود sendMessage به متد POST یک درخواست fetch با استفاده از.

Scanned with CamScanner

در بدنه درخواست، دو مقدار اصلی قرار می‌گیرند:
• chat_id: شناسه گفت‌وگوی مقصد
• text: متن پاسخ

۷.۷. مدیریت دستور شروع

```
if (userText === "/start" ||
   userText === "/start@YumioAIbot") {
```

اگر کاربر دستور شروع را ارسال کند، ربات پیام خوشامدگویی نمایش می‌دهد.
و راهنمای اولیه ارسال مواد غذایی است Yumio این پیام شامل معرفی.

۷.۸. مدیریت دستور راهنما

```
if (userText === "/help") {
```

این شرط دستور راهنما را بررسی می‌کند.
در صورت دریافت آن، ربات نمونه‌هایی از نحوه استفاده و قابلیت‌های خود را برای کاربر ارسال می‌کند.

۷.۹. نمایش وضعیت تایپ

```
await fetch(
  `https://api.telegram.org/bot${env.TELEGRAM_BOT_TOKEN}/sendChatAction`,
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      chat_id: chatId,
      action: "typing"
    })
  }
);
```

این بخش باعث می‌شود کاربر هنگام پردازش درخواست توسط هوش مصنوعی، وضعیت تایپ کردن ربات را مشاهده کند.

۷.۱۰. تعریف System Prompt

System Prompt مجموعه دستورالعمل‌هایی است که نقش و رفتار هوش مصنوعی را مشخص می‌کند.
این دستورالعمل‌ها شامل موارد زیر هستند Yumio در پروژه:
• پاسخ‌گویی به زبان فارسی
• تحلیل مواد غذایی اعلام‌شده
• پیشنهاد سه غذای مناسب
• توجه به مواد اولیه ضروری
• اعلام زمان تقریبی آماده‌سازی

Scanned with CamScanner

• پرسیدن سؤال در صورت کافی نبودن اطلاعات
• ارائه دستور پخت مرحله‌به‌مرحله
• رعایت حساسیت‌ها و محدودیت‌های غذایی
• استفاده از لحن دوستانه و پاسخ‌های مختصر
باعث می‌شود مدل به‌جای پاسخ‌گویی عمومی، در نقش یک دستیار آشپزی فعالیت کند System Prompt.

۷.۱۱. ارسال درخواست به هوش مصنوعی

```
const result = await env.AI.run(
  "@cf/meta/llama-3.1-8b-instruct-fast",
  {
    messages: [
      {
        role: "system",
        content: systemPrompt
      },
      {
        role: "user",
        content: userText
      }
    ]
  }
);
```

این بخش هسته هوشمند پروژه است.
دو نوع پیام برای مدل ارسال می‌شود:
• System Message: دستورالعمل‌های کلی و نقش Yumio را مشخص می‌کند.
• User Message: متن واقعی ارسال‌شده توسط کاربر را در اختیار مدل قرار می‌دهد.
مدل با توجه به هر دو پیام، پاسخ مناسب را تولید می‌کند.

۷.۱۲. دریافت پاسخ مدل

```
const answer =
  result.response ||
  "متأسفم، نتونستم پیشنهاد مناسبی پیدا کنم.";
```

در این بخش پاسخ تولیدشده توسط مدل استخراج می‌شود.
اگر پاسخ در دسترس نباشد، یک پیام جایگزین برای کاربر در نظر گرفته شده است.

۷.۱۳. ارسال پاسخ نهایی

```
await sendMessage(answer.substring(0, 4000));
```

برای کاربر ارسال می‌شود sendMessage پاسخ هوش مصنوعی از طریق تابع.
محدود کردن متن به ۴۰۰۰ کاراکتر برای جلوگیری از ارسال پیام بیش از حد طولانی در این نسخه در نظر گرفته شده است.

Scanned with CamScanner

۷.۱۴. مدیریت خطا

```
catch (error) {
  console.error("Yumio Error:", error);
  return new Response(
    "Internal Server Error",
    { status: 500 }
  );
}
```

ثبت می‌کند Worker در صورت بروز خطا، برنامه آن را در گزارش‌های.
این قابلیت برای شناسایی مشکلاتی مانند خطای ارتباط با مدل هوش مصنوعی یا مشکلات اجرای برنامه کاربرد دارد.

۸. مفاهیم رایانش ابری استفاده‌شده در پروژه

۸.۱. Serverless Computing

است Serverless Computing یکی از مهم‌ترین مفاهیم به‌کاررفته در پروژه، رایانش بدون سرور یا.
در این معماری، توسعه‌دهنده نیازی به خرید، نصب و مدیریت مستقیم سرور ندارد.
کد برنامه روی زیرساخت ابری ارائه‌دهنده سرویس اجرا می‌شود.
وظیفه اجرای کد را بر عهده دارد Yumio، Cloudflare Workers در پروژه.

۸.۲. Cloud Computing

پردازش پیام‌ها و اجرای مدل هوش مصنوعی از طریق سرویس‌های ابری انجام می‌شود.
نیازی به داشتن سخت‌افزار قدرتمند یا نصب مدل هوش مصنوعی روی Yumio بنابراین کاربر برای استفاده از دستگاه خود ندارد.

۸.۳. API

امکان ارتباط میان سرویس‌های مختلف را فراهم می‌کند API رابط برنامه‌نویسی کاربردی یا.
در این پروژه:

- برای ارتباط با تلگرام Telegram Bot API
- برای اجرای مدل هوش مصنوعی Cloudflare Workers AI API

استفاده شده‌اند.

۸.۴. Webhook

امکان ارسال خودکار رویدادها از یک سرویس به سرویس دیگر را فراهم می‌کند Webhook.
ارسال می‌کند Worker تلگرام با دریافت پیام جدید، اطلاعات آن را به.
برای ساختار فعلی پروژه مناسب است Polling این روش نسبت به بررسی مداوم پیام‌ها از طریق.

۸.۵. Binding

است Cloudflare به منابع و سرویس‌های Worker روشی برای اتصال مستقیم Binding.
استفاده شده است Workers AI برای دسترسی به AI با نام Binding، در پروژه.

Scanned with CamScanner

۸.۶. Secret Management

نگهداری می‌شوند. Cloudflare در تنظیمات Secret اطلاعات حساس مانند توکن ربات به‌صورت
این روش بهتر از قرار دادن توکن در متن کد است؛ زیرا از نمایش مستقیم مقدار آن در سورس برنامه جلوگیری می‌کند.

نحوه آزمایش و ارزیابی پروژه ۹.

برای اطمینان از عملکرد صحیح ربات، آزمایش‌های مختلفی انجام می‌شود.

شماره	نوع آزمایش	ورودی نمونه	نتیجه مورد انتظار
۱	شروع ربات	/start	نمایش پیام خوشامدگویی
۲	راهنمای ربات	/help	نمایش راهنمای استفاده
۳	پیشنهاد غذا	تخم‌مرغ، سیب‌زمینی و گوجه	پیشنهاد سه غذای مناسب
۴	محدودیت زمانی	فقط ۱۰ دقیقه وقت دارم	پیشنهاد غذای سریع
۵	سلیقه غذایی	غذای ایرانی و سبک	پیشنهاد متناسب با سلیقه
۶	دستور پخت	طرز تهیه غذای اول	توضیح مراحل پخت
۷	حساسیت غذایی	حساسیت به لبنیات	پرهیز از پیشنهاد مواد لبنی
۸	مواد اولیه محدود	فقط ماکارونی دارم	پیشنهاد مناسب یا پرسیدن سؤال
۹	درخواست خارج از موضوع	راهنمای یادگیری زبان	پاسخ مناسب یا هدایت به موضوع آشپزی

معیارهای ارزیابی:

- صحت دریافت پیام
- سرعت پاسخ‌گویی
- ارتباط پاسخ با مواد غذایی اعلام‌شده
- رعایت محدودیت‌های کاربر
- کیفیت پیشنهادهای غذایی
- قابل فهم بودن دستور پخت
- مدیریت درخواست‌های نامرتبط

مشکلات فنی و روش رفع آن‌ها ۱۰.

در زمان پیاده‌سازی پروژه، برخی مشکلات فنی مشاهده شد.

مشکل اتصال وب‌هوک ۱۰.۱

در ابتدا تنظیم Webhook به‌درستی انجام نشده بود.
برقرار شد Worker ارتباط تلگرام با setWebhook، و استفاده از متد URL با بررسی ساختار.

مشکل اجرای مدل هوش مصنوعی ۱۰.۲

پس از برقراری ارتباط تلگرام، ربات پیام‌ها را دریافت می‌کرد اما در زمان اجرای مدل پاسخ نمی‌داد.
خطای مربوط به دستور Cloudflare در گزارش‌های env.AI.run() مشاهده شد.
پاسخ‌گویی هوش مصنوعی برقرار، Worker با تغییر شناسه مدل به نسخه انتخاب‌شده در کد فعلی و انتشار مجدد شد.

بررسی گزارش‌های خطا ۱۰.۳

برای بررسی درخواست‌ها و خطاهای زمان اجرا استفاده Cloudflare در Logs و Observability از بخش

Scanned with CamScanner

شد.
این بخش در عیب‌یابی ارتباط با سرویس هوش مصنوعی نقش مهمی داشت.

۱۱. مزایای پروژه

- استفاده آسان از طریق تلگرام
- عدم نیاز به نصب نرم‌افزار اختصاصی
- استفاده از زیرساخت ابری
- عدم نیاز به مدیریت سرور فیزیکی
- استفاده از هوش مصنوعی برای درک زبان طبیعی
- امکان توسعه قابلیت‌های جدید
- هزینه و پیچیدگی اولیه کمتر نسبت به راه‌اندازی یک سرور اختصاصی و مدل محلی

۱۲. محدودیت‌های نسخه فعلی

با وجود عملکرد مناسب نسخه اولیه، پروژه محدودیت‌هایی دارد:

- تاریخچه مکالمات کاربران ذخیره نمی‌شود.
- ربات حافظه دائمی برای نگهداری مواد غذایی موجود در خانه ندارد.
- پیشنهادهای غذایی ممکن است همیشه از نظر دستور پخت کاملاً دقیق نباشند.
- اطلاعات تغذیه‌ای و کالری غذاها به‌صورت تخصصی اعتبارسنجی نمی‌شوند.
- نسخه فعلی فاقد پنل مدیریتی مستقل است.
- امکان ثبت و مدیریت فهرست خرید وجود ندارد.
- پاسخ مدل به کیفیت و دسترس‌پذیری سرویس هوش مصنوعی وابسته است.
- در نسخه فعلی، بررسی تخصصی ایمنی مواد غذایی و حساسیت‌ها انجام نمی‌شود و کاربر باید اطلاعات حساسیت خود را با دقت اعلام کند.

۱۳. پیشنهادهایی برای توسعه آینده

برای ارتقای پروژه می‌توان قابلیت‌های زیر را اضافه کرد:

- استفاده از پایگاه داده Cloudflare D1 برای ذخیره اطلاعات کاربران
- ذخیره تاریخچه مکالمات و مواد غذایی موجود در خانه
- ایجاد پروفایل اختصاصی برای هر کاربر
- اضافه کردن امکان ثبت غذاهای موردعلاقه
- طراحی سیستم برنامه‌ریزی وعده‌های غذایی هفتگی
- محاسبه تقریبی کالری و ارزش غذایی با استفاده از منابع معتبر
- اضافه کردن قابلیت مدیریت فهرست خرید
- امکان پیشنهاد غذا بر اساس بودجه کاربر
- ایجاد رابط کاربری گرافیکی و دکمه‌های تعاملی تلگرام
- اضافه کردن سیستم امتیازدهی به غذاهای پیشنهادی

۱۴. نتیجه‌گیری

پروژه Yumio نمونه‌ای کاربردی از ترکیب رایانش ابری و هوش مصنوعی در قالب یک ربات تلگرامی است. در این پروژه، Cloudflare Workers برای اجرای منطق برنامه، Workers AI برای پردازش زبان طبیعی و

Scanned with CamScanner

برای برقراری ارتباط با کاربران استفاده شده‌اند Telegram Bot API.
این ترکیب نشان می‌دهد که می‌توان بدون راه‌اندازی سرور اختصاصی و بدون نصب مدل هوش مصنوعی روی رایانه شخصی، یک دستیار هوشمند طراحی و اجرا کرد.
در نسخه فعلی قادر است پیام‌های کاربران را دریافت کند، درخواست‌های مرتبط با آشپزی را تحلیل کند و Yumio پیشنهادهای غذایی ارائه دهد.
همچنین ساختار پروژه به‌گونه‌ای است که در نسخه‌های آینده می‌توان امکانات بیشتری مانند ذخیره اطلاعات کاربران، مدیریت مواد غذایی و برنامه‌ریزی وعده‌های غذایی را به آن اضافه کرد.
این پروژه تجربه‌ای عملی از طراحی، پیاده‌سازی و استقرار یک برنامه هوشمند مبتنی بر خدمات رایانش ابری است.

۱۵. منابع و مستندات

• Cloudflare Workers Documentation: https://developers.cloudflare.com/workers/
• Cloudflare Workers AI Documentation: https://developers.cloudflare.com/workers-ai/
• Cloudflare Workers Bindings: https://developers.cloudflare.com/workers/runtime-apis/bindings/
• Telegram Bot API: https://core.telegram.org/bots/api
• Telegram Bot Features: https://core.telegram.org/bots/features

پایان مستند فنی پروژه Yumio

Scanned with CamScanner